STELLAR COMMUNITY FUND · ROUND 46 · BUILD AWARD, OPEN TRACK

# RWAXI · Architecture on Stellar

The twelve views of the Archi model, from system context to data flows, and what each one settles.

Carlos Serna, technical lead · Nekot, technical provider to RWAXI Corp.
Version 0.2 · 30 September 2026 · Draft for team review

STARTING POINT

## The decision everything else comes from

RWAXI issues and administers real-world-asset security tokens on Stellar. The first vehicle is Avista Beach Tokenization Fund I, villas and plots in Los Córdobas (Córdoba, Colombia), offered under US Regulation S.

> Transfer rules run inside the token contract, not in a database the operator can edit.

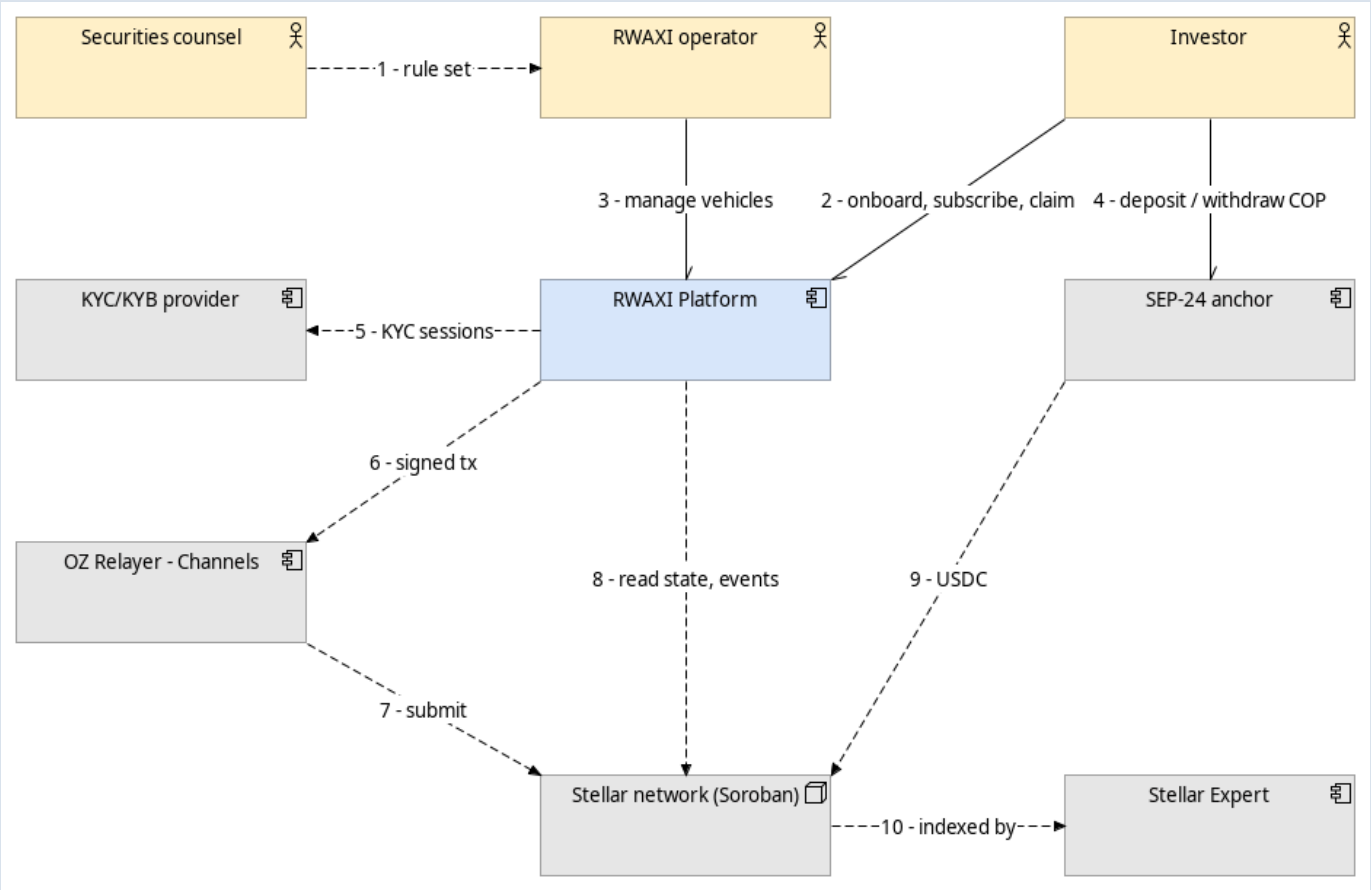
Every transfer, mint and burn checks the identity of both parties and runs the vehicle's compliance modules in the same transaction. If anything fails, everything reverts. So it doesn't matter where the call comes from: our API, another wallet or a direct contract call all go through the same check.

| What the Build Award asks for | How we answer it | Views |
|---|---|---|
| Clear Stellar integration | OpenZeppelin stellar-contracts RWA, identity and compliance modules, passkey smart accounts, OZ Relayer for fees, USDC and a SEP-24 anchor. | C4-2, C4-3 |
| Ambitious but scoped | Five contracts of our own. The rest is OpenZeppelin code we only configure. | C4-3 |
| Verifiable deliverables | The spike's blocked and allowed transfers stay on Stellar Expert with public hashes. | Flow 1, DFD 2 |
| Security by design | DFDs with six trust boundaries as input to the STRIDE threat model. | DFD 1a, 1b |

Colors in every view: people · what we build or configure (OpenZeppelin and platform) · RWAXI's own contracts · external systems

VIEW C4-1

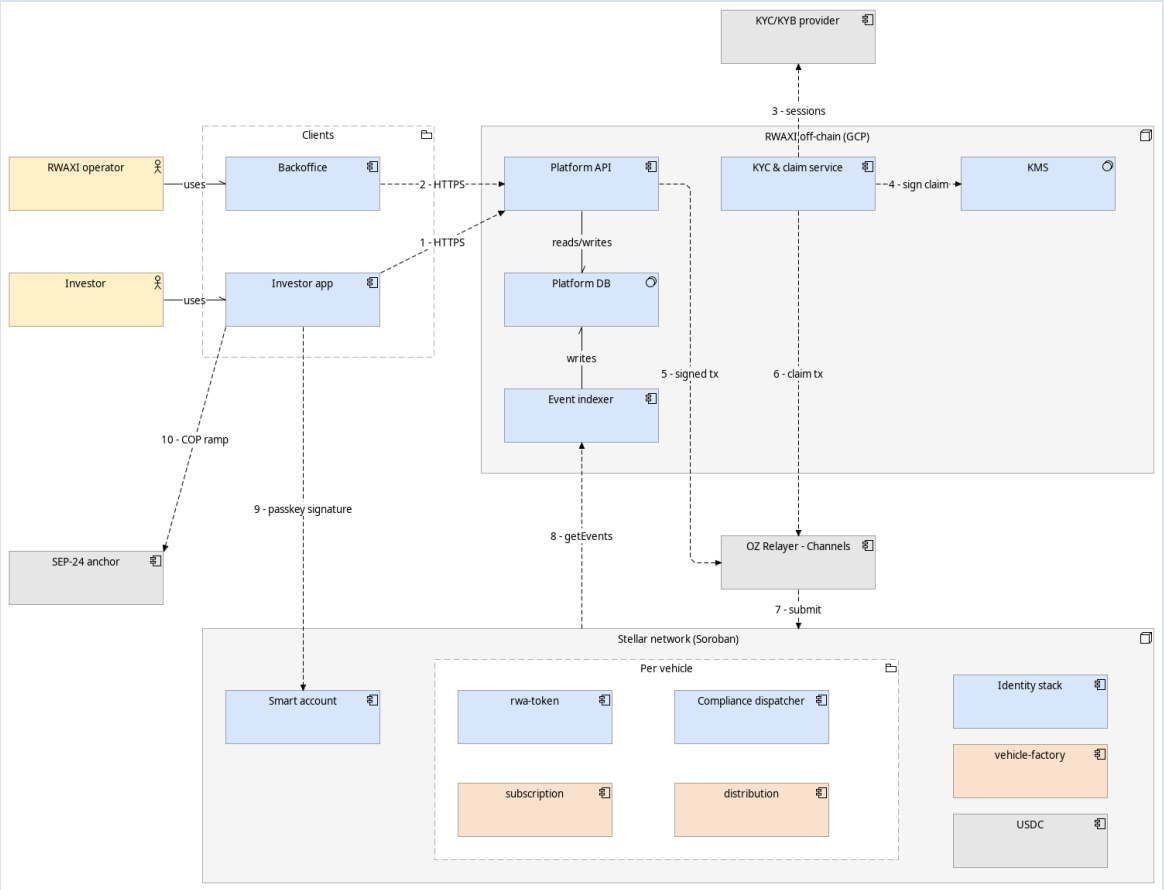
# System context




Three people sit around the platform: the investor, the RWAXI operator and the securities counsel. Counsel never touches the system. They hand the operator a written rule set and the operator turns it into module configuration. Anything that moves money or ownership ends on Stellar, and the result is public on Stellar Expert. KYC and the COP ramp stay with licensed providers on purpose. We don't want to hold identity documents or fiat.

VIEW C4-2

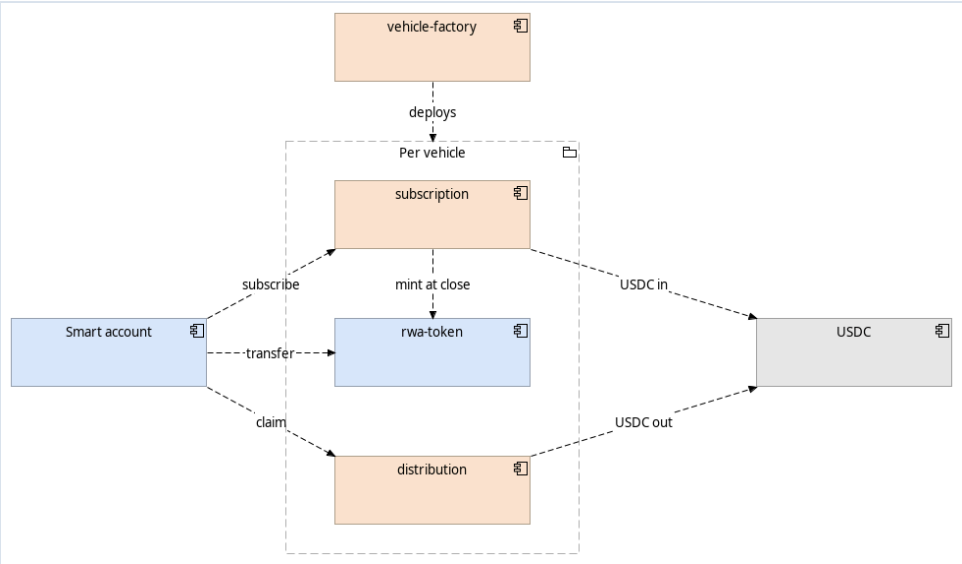
# Containers

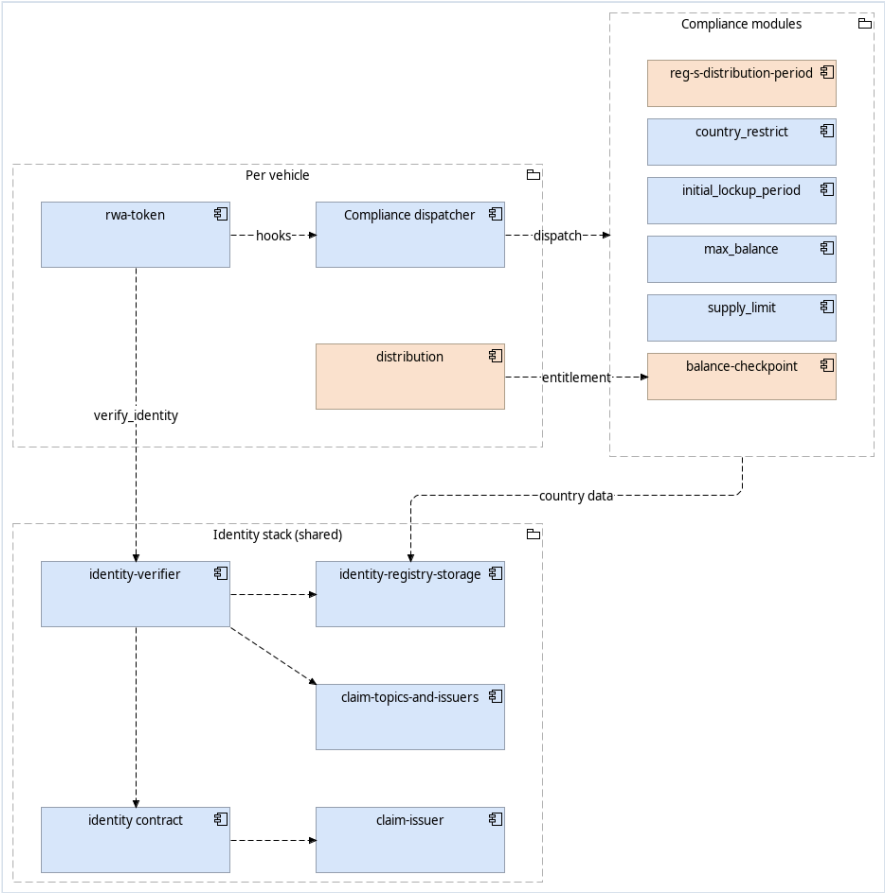



Three blocks: the clients (investor app and backoffice), what runs in our GCP project, and the contracts on Soroban. The investor signs with a passkey on the phone; the API builds and simulates transactions but never holds an investor key. Only the KYC service can use the claim-signing key in KMS. Everything goes out through the OZ Relayer, so the investor never needs to buy XLM. The indexer fills Postgres for screens and reports; the ledger is the source of truth.

VIEW C4-3

# On-chain components

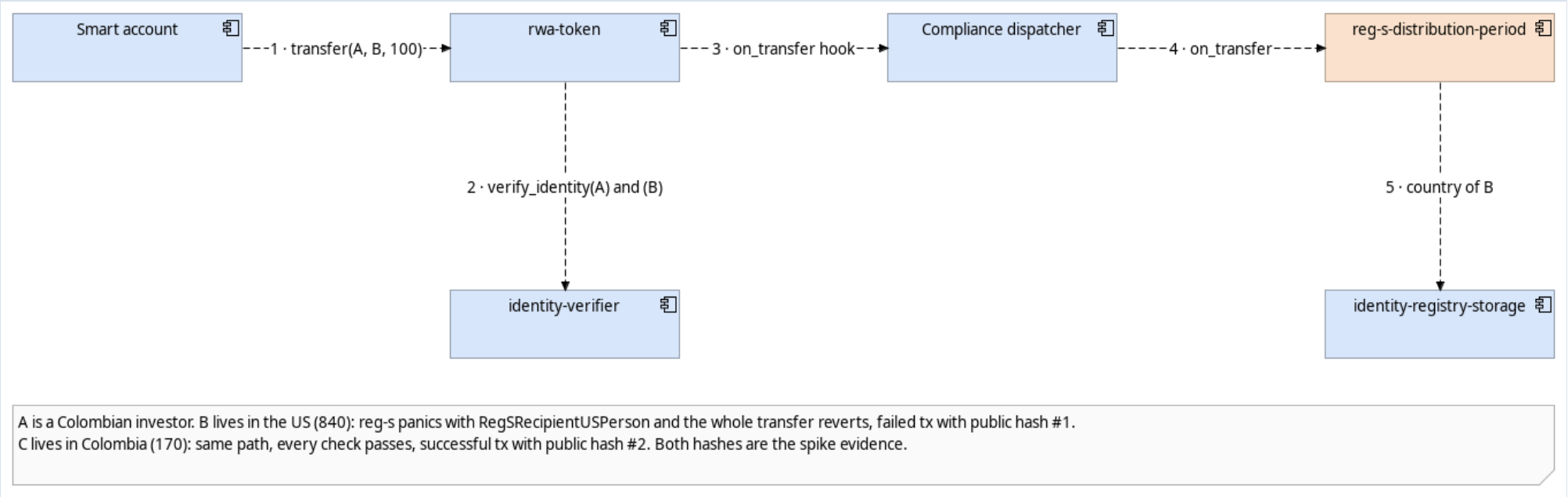



We split this level in two because with every contract in one picture the lines kept crossing. On the left, what each vehicle gets: the factory deploys the set and the investor's smart account subscribes, transfers and claims. On the right, what makes it a security token: the token verifies both identities and fires its hooks, and the dispatcher runs the vehicle's modules. We write two modules, the Reg S one and the balance checkpoint; the rest comes from OpenZeppelin.

FLOW 1

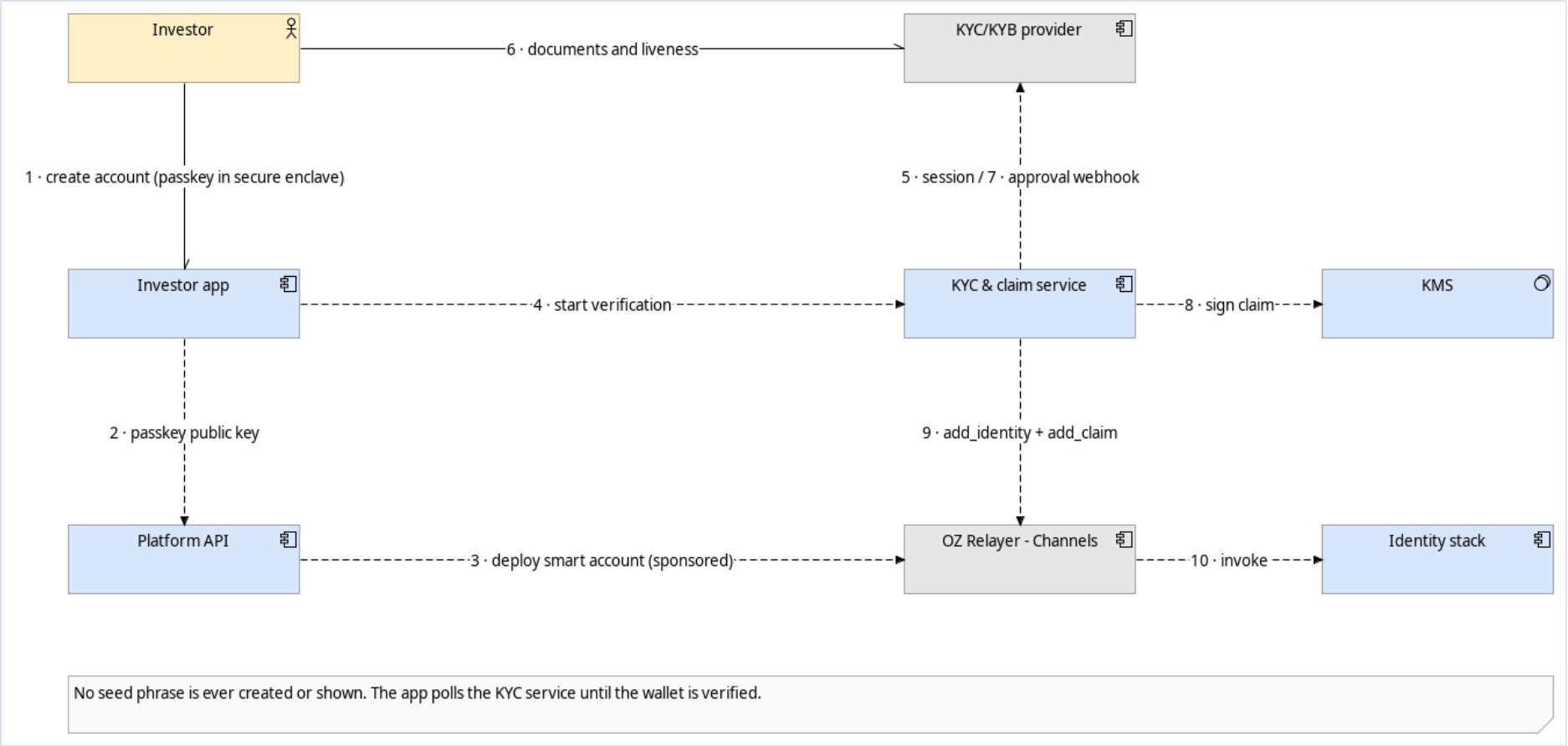
## Compliance-checked transfer (what the spike proves)




A Colombian investor tries to send tokens to a US resident during the Reg S period. The module reads the recipient's country, rejects, and the whole transaction reverts; the failed attempt keeps a public hash. The same transfer to a Colombian goes through. Those two hashes are the core evidence of the application.

FLOW 2

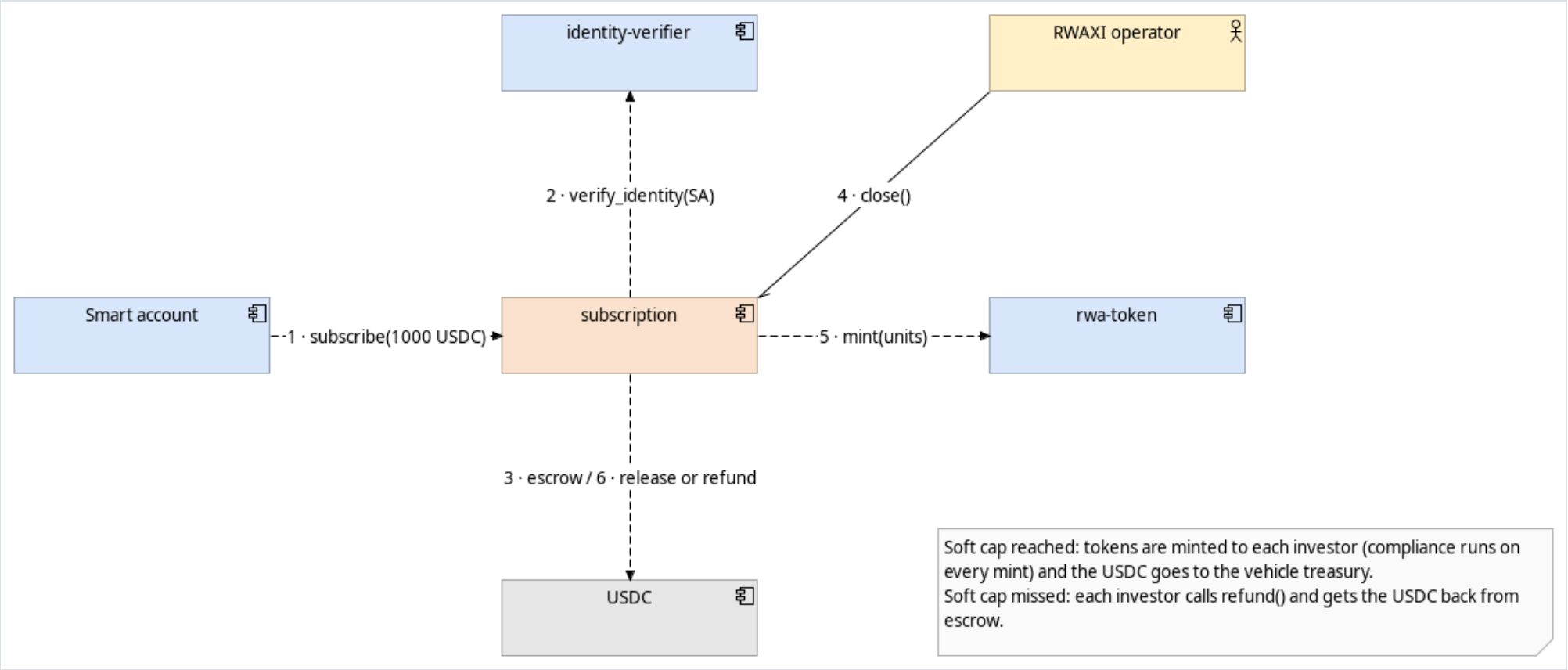
# Onboarding: passkey, KYC and on-chain claim




The account starts with a passkey on the phone and the smart account is deployed with a sponsored fee. Documents go straight from the phone to the KYC provider; we get the decision, sign the claim in KMS and register it in the identity stack. The investor never sees a seed phrase.

FLOW 3

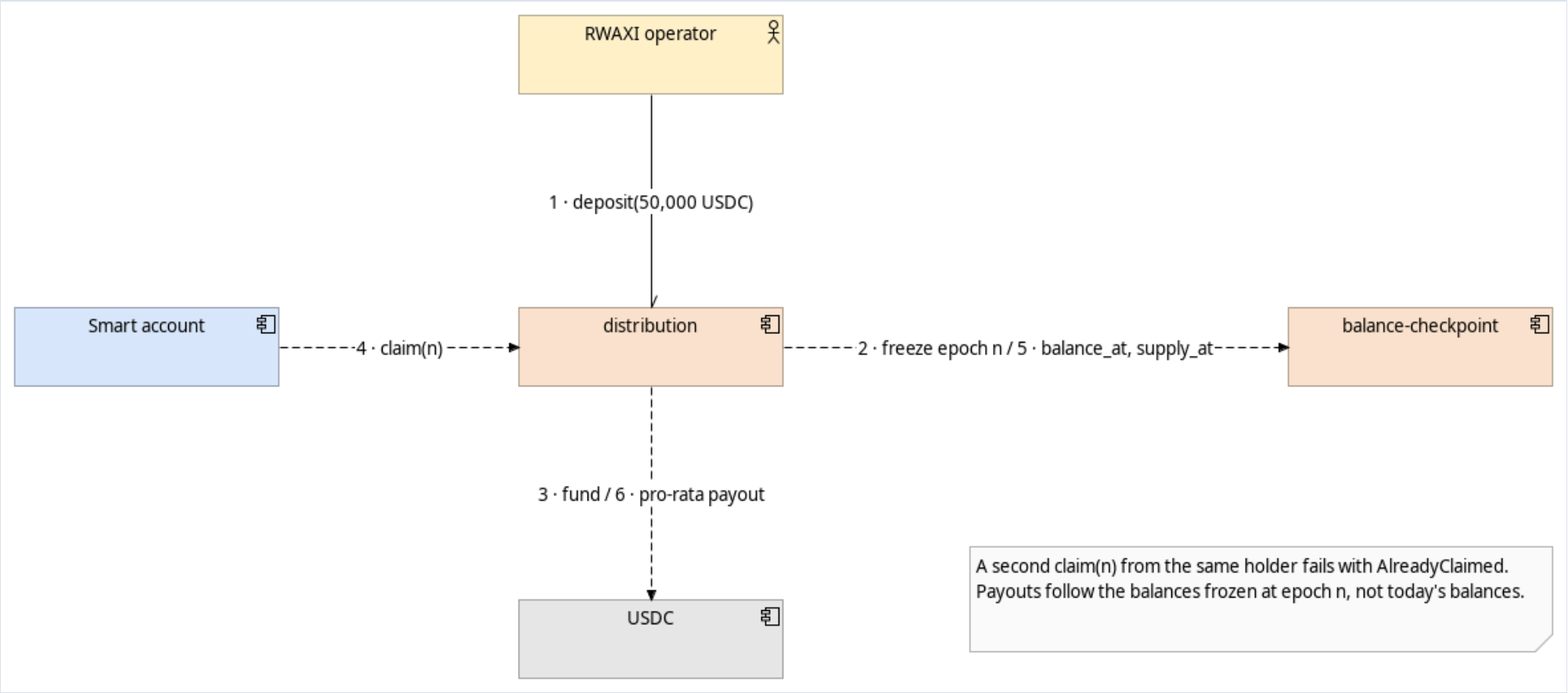
## Subscription (tranche 2)




The subscription contract only takes USDC from verified accounts and holds it in escrow. At close, if the soft cap was reached it mints the tokens (each mint goes through compliance) and releases the USDC to the vehicle. If not, every investor can get a refund without depending on anyone.

FLOW 4

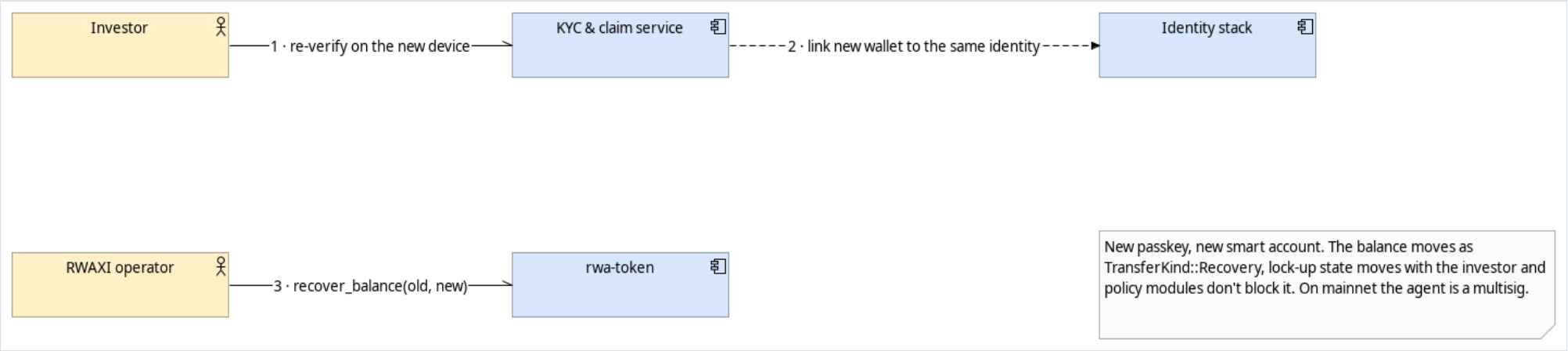
## Distribution (tranche 2)




The operator deposits the USDC, the checkpoint freezes balances for that epoch and each holder claims a pro-rata share. Paying on frozen balances stops someone from buying the day before a payout just to collect it.

FLOW 5

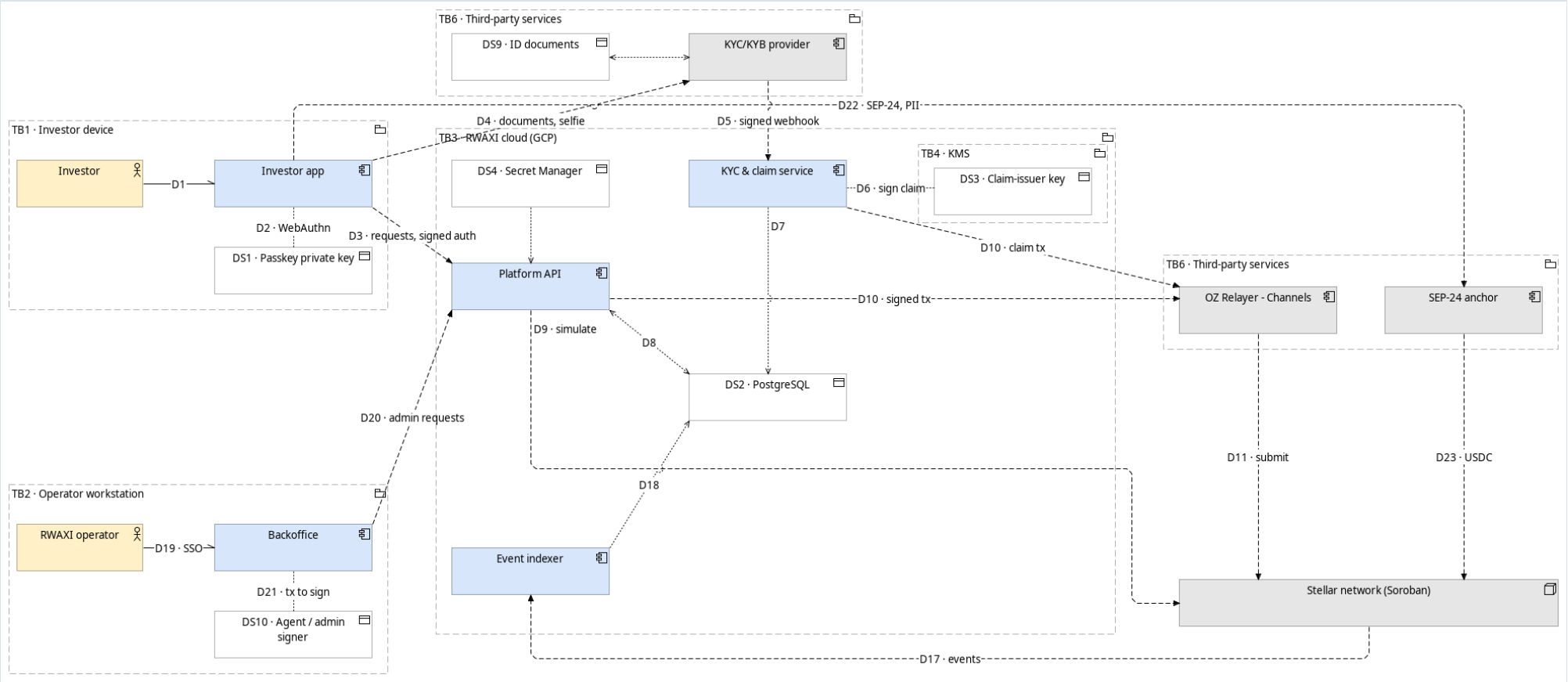
# Lost device recovery




Losing the phone doesn't lose the investment. The investor verifies again on the new device, the new wallet is linked to the same identity and the agent moves the balance with recover_balance. On mainnet that agent is a multisig, not one person.

DFD 1A

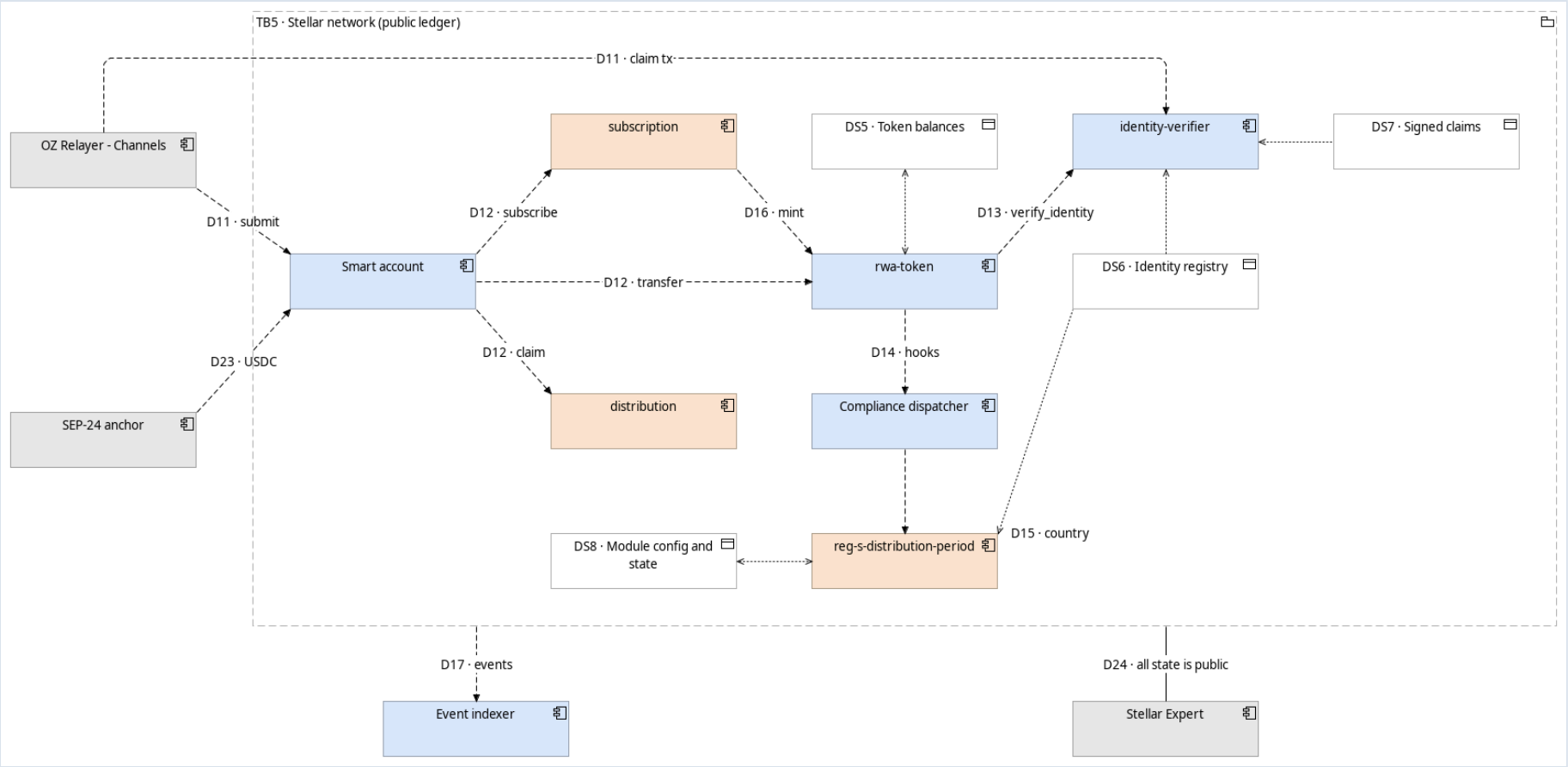
# Off-chain data flows




We follow Stellar's threat-modeling guide. Each dashed box is a trust boundary (TB1 to TB6) and each arrow a data flow (D1 to D23). The most valuable asset off-chain is the claim-signing key (DS3), isolated in KMS inside TB4. Identity documents (DS9) never go through our servers.

DFD 1B

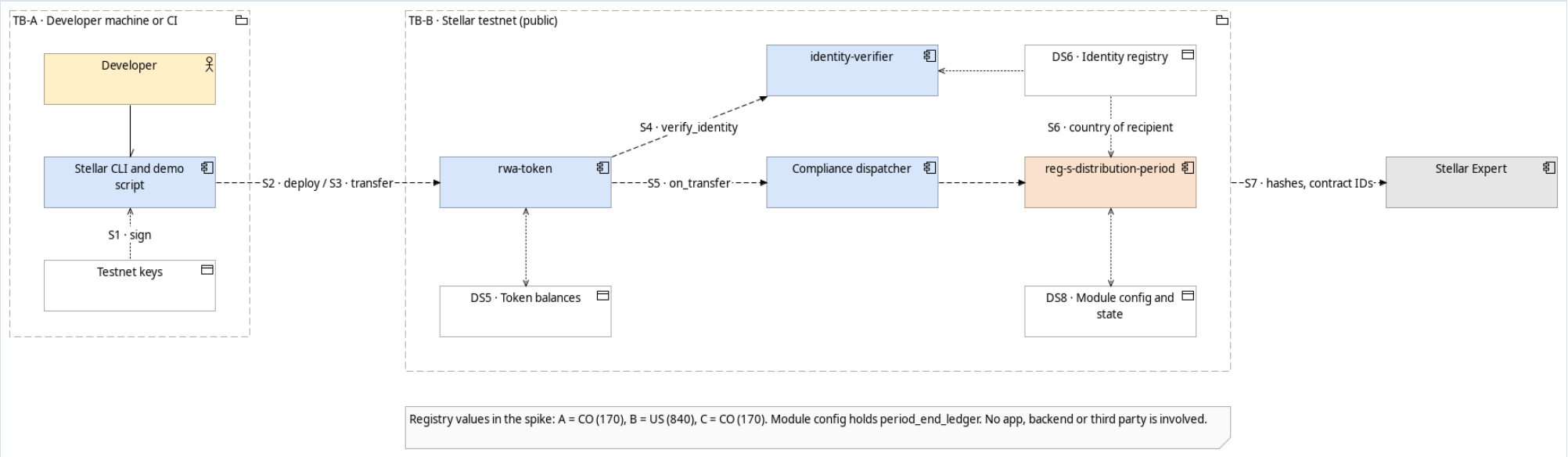
# On-chain data flows




Everything in TB5 is public. That's why the design rule is that no data identifying a person by name or document reaches the chain: there are only balances, ISO country codes and signed claims, all pseudonymous.

DFD 2

# Spike data flows on testnet




The spike has no app, backend or third parties. A developer runs it with the Stellar CLI from a laptop or CI. It's the minimum scope needed to produce the two evidence hashes.

KEEPING IT CURRENT

## One source for every diagram

All twelve views come from the same Archi model (`docs/7-model/rwaxi-architecture.archimate`), so they share the same names. When the architecture changes, change the model first and export the images again; the PNGs are never edited by hand.

The reasoning behind each decision is in ADR-001 to ADR-007 in the repository. The tables of data flows, data stores and trust boundaries are in `docs/5-security.md`.